LAPORAN TUGAS 7: CRUD TRANSAKSI DAN HAK AKSES

Tabel Transactions - Sistem Informasi Penjualan Buku (Book Sales)

Nama : Muhamad Aditia
Nim : 0110224213
Rombel : TI02 STT-NF

1. Tujuan

Membuat fitur CRUD untuk tabel Transaksi melalui API, dengan data yang diambil dari relasi Model (customer dan buku), relasi antar tabel yang diatur dengan foreignId pada migration, serta hak akses yang diatur pada routing: Create, Update, dan Show hanya untuk customer yang sudah login, sedangkan Read All dan Destroy hanya untuk admin.

2. Bisnis Proses Transaksi

Karakteristik	Pada aplikasi ini
Tujuan yang jelas	Customer dapat membeli buku dan transaksinya tercatat, sedangkan admin dapat memantau dan menghapus transaksi.
Input dan output	Input: book_id dari customer. Output: transaksi berisi order_number, customer, buku, dan total_amount.
Serangkaian aktivitas	Login, validasi book_id, pengambilan harga buku, penyimpanan transaksi, lalu respons JSON.
Aturan akses	Dijaga oleh middleware: auth:sanctum untuk login, CustomerMiddleware untuk customer, dan AdminMiddleware untuk admin.

3. Pemenuhan Instruksi Tugas

Poin	Instruksi	Pelaksanaan
2	CRUD tabel Transaksi	TransactionController dengan method index, store, show, update, dan destroy.
3.1	Mengambil data dari relasi Model	Model Transaction memiliki relasi customer() dan book() (belongsTo). Data diambil dengan with().
3.2	Relasi di migration dengan foreignId	customer_id dan book_id dibuat dengan foreignId()->constrained().
4.1	Create, Update, Show hanya customer login	Route store, update, dan show berada di grup auth:sanctum dan CustomerMiddleware.
4.2	Read All dan Destroy hanya admin	Route index dan destroy berada di grup auth:sanctum dan AdminMiddleware.
5	Testing dengan Postman	Setiap aturan akses diuji dengan Postman (lihat bagian 7).
6	Push ke GitHub	Proyek diunggah ke folder book-sales-7 (lihat bagian 8).

4. Migration dan Relasi

Tabel transactions mengikuti desain Brief. Kolom customer_id merujuk ke tabel users dan book_id merujuk ke tabel books menggunakan foreignId.

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('transactions', function (Blueprint $table) {
            $table->id();
            $table->string('order_number')->unique();
            // foreignId: relasi ke tabel users (customer) dan books
            $table->foreignId('customer_id')->constrained('users');
            $table->foreignId('book_id')->constrained('books');
            $table->decimal('total_amount', 10, 2);
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('transactions');
    }
};
```

Pada Model Transaction, relasi didefinisikan sehingga data customer dan buku dapat ikut ditampilkan.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Transaction extends Model
{
    protected $table = 'transactions';

    protected $fillable = ['order_number', 'customer_id', 'book_id', 'total_amount'];

    // Relasi: satu transaksi dimiliki oleh satu customer (tabel users)
    public function customer(): BelongsTo
    {
        return $this->belongsTo(User::class, 'customer_id');
    }

    // Relasi: satu transaksi untuk satu buku
    public function book(): BelongsTo
    {
        return $this->belongsTo(Book::class);
    }
}
```

5. Routing (routes/api.php)

Hak akses diatur dengan mengelompokkan route berdasarkan middleware.

```
<?php

use App\Http\Controllers\AuthController;
```

```
use App\Http\Controllers\AuthorController;
use App\Http\Controllers\GenreController;
use App\Http\Controllers\TransactionController;
use App\Http\Middleware\AdminMiddleware;
use App\Http\Middleware\CustomerMiddleware;
use Illuminate\Support\Facades\Route;


// Semua route di file ini otomatis berawalan /api

// ---------------------------------------------------------------
// 1. Autentikasi (bebas diakses)
// ---------------------------------------------------------------
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);


// Dibutuhkan oleh middleware auth: jika belum login, Laravel mengarahkan ke route
// bernama "login". Route ini membalas JSON 401 supaya tidak muncul error HTML.
Route::get('/login', function () {
    return response()->json([
        'success' => false,
        'message' => 'Silakan login terlebih dahulu',
        'data'    => null,
    ], 401);
})->name('login');


// ---------------------------------------------------------------
// 2. Genre dan Author: Read All dan Show bisa diakses semua orang
// ---------------------------------------------------------------
Route::apiResource('genres', GenreController::class)->only(['index', 'show']);
Route::apiResource('authors', AuthorController::class)->only(['index', 'show']);


// ---------------------------------------------------------------
// 3. Wajib login (token)
// ---------------------------------------------------------------
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/me', [AuthController::class, 'me']);
    Route::post('/logout', [AuthController::class, 'logout']);


    // ---------------------------------------------------------------
    // 4. Khusus ADMIN
    // ---------------------------------------------------------------
    Route::middleware(AdminMiddleware::class)->group(function () {
        // Genre dan Author: Create, Update, Destroy
        Route::apiResource('genres', GenreController::class)->except(['index', 'show']);
        Route::apiResource('authors', AuthorController::class)->except(['index', 'show']);


        // Transaksi: Read All dan Destroy
        Route::apiResource('transactions', TransactionController::class)->only(['index', 'destroy']);
    });


    // ---------------------------------------------------------------
    // 5. Khusus CUSTOMER (yang sudah login)
    // ---------------------------------------------------------------
    Route::middleware(CustomerMiddleware::class)->group(function () {
        // Transaksi: Create, Update, Show
        Route::apiResource('transactions', TransactionController::class)->only(['store', 'update', 'show']);
    });
});
```

6. Controller

Customer hanya mengirim book_id. Customer yang menyimpan transaksi diambil dari user yang login, dan total_amount diambil dari harga buku, sehingga tidak dapat dimanipulasi oleh pengguna. Pada Show dan Update, customer hanya boleh mengakses transaksi miliknya sendiri (jika bukan miliknya, status 403), dan data yang tidak ada menghasilkan status 404.

```
    // index(), update(), dan destroy() mengikuti pola yang sama
```

```
    // POST /api/transactions : Create (hanya customer)
    public function store(Request $request): JsonResponse
    {
        $validator = Validator::make($request->all(), [
            'book_id' => 'required|integer|exists:books,id',
        ]);

        if ($validator->fails()) {
            return $this->validationError($validator);
        }

        $book = Book::find($request->book_id);

        $transaction = Transaction::create([
            'order_number' => 'TRX-' . now()->format('Ymd') . '-' . strtoupper(Str::random(6)),
            'customer_id'  => $request->user()->id, // diambil dari user yang login
            'book_id'      => $book->id,
            'total_amount' => $book->price,
        ]);

        $transaction->load(['customer:id,name,email', 'book:id,title,price']);

        return response()->json([
            'success' => true,
            'message' => 'Transaksi berhasil dibuat',
            'data'    => $transaction,
        ], 201);
    }

    // GET /api/transactions/{transaction} : Show (hanya customer pemilik transaksi)
    public function show(Request $request, $id): JsonResponse
    {
        $transaction = Transaction::with(['customer:id,name,email', 'book:id,title,price'])->find($id);

        if (!$transaction) {
            return $this->notFound();
        }

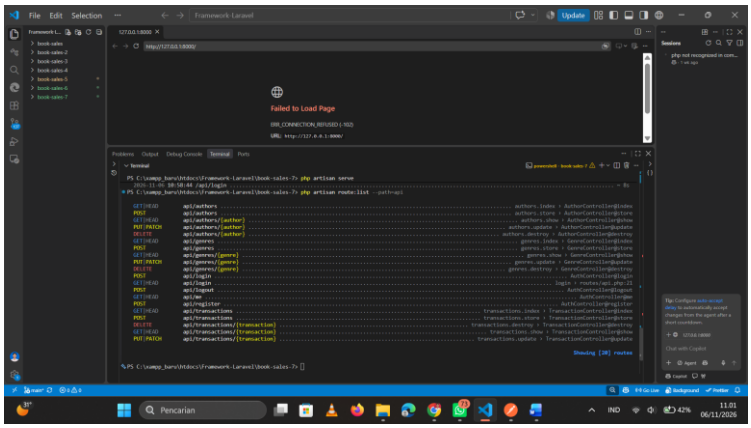
        if ($transaction->customer_id !== $request->user()->id) {
            return $this->forbidden();
        }

        return response()->json([
            'success' => true,
            'message' => 'Detail data transaksi',
            'data'    => $transaction,
        ]);
    }
}
```

7. Pengujian dengan Postman

Akun yang dipakai: admin@example.com, customer@example.com, dan customer2@example.com (password password123). Token diperoleh dari POST /api/login dan dipasang pada Authorization dengan tipe Bearer Token.

Method	URL	Akses	Status
POST	/api/transactions	Token customer (book_id: 1)	201 Created
GET	/api/transactions/{id}	Token customer pemilik	200 OK
GET	/api/transactions/{id}	Token customer lain	403 Forbidden
PUT	/api/transactions/{id}	Token customer pemilik	200 OK
GET	/api/transactions	Token customer	403 Forbidden
GET	/api/transactions	Token admin	200 OK
POST	/api/transactions	Tanpa token	401 Unauthorized
DELETE	/api/transactions/{id}	Token customer	403 Forbidden
DELETE	/api/transactions/{id}	Token admin	200 OK



8. GitHub

Link repository: https://github.com/muhammadaditia433/Framework-Laravel/tree/main/book-sales-7

File yang dikumpulkan: routes/api.php.

9. Kesimpulan

CRUD transaksi berhasil dibuat dengan relasi ke customer dan buku. Pengaturan hak akses lewat middleware pada routing memastikan hanya customer yang dapat membuat, mengubah, dan melihat transaksinya sendiri, sedangkan melihat seluruh transaksi dan menghapus hanya dapat dilakukan oleh admin.